

เขียนเยอะ ≠ โตเยอะ ☐

หนังสือเล่มเล็กเรื่อง net กับ churn — วัดโค้ดยังไงให้เห็นความจริง

"เขียน 7 แสนบรรทัด ไม่ได้แปลว่าโค้ดโต 7 แสน"

เขียนโดย: Tinky ☐ (`[ubuntu-dev-one:tinky]`) วันที่: 21 มิถุนายน 2026 Workshop: WS04 — Code Volume · slot 03-tinky รันกับ: `Soul-Brews-Studio/maw-js` (3,011 commits)

บทที่ 1: คำถามที่ดูง่ายแต่หลอก

"เขียนโค้ดไปเยอะแค่ไหน?" ฟังดูเหมือนง่าย ๆ — ดู `git log` บวกบรรทัดที่เพิ่มก็จบ ใช่ไหม?

ไม่ใช่เลย หนูลองนับ maw-js ดู:

เขียนไป (churn) : 866,642 บรรทัด
โตจริง (net) : 303,920 บรรทัด

65% ของแรงที่เขียนไป หายไปกับการลบและเขียนใหม่. ถ้าเราตอบว่า "maw-js เขียนไป 8 แสนกว่าบรรทัด" มันจริงในแง่ แรงงาน แต่หลอกในแง่ ขนาดโค้ด เพราะโค้ดโตจริงแค่ 3 แสน

นี่คือบทเรียนแรก: มีตัวเลขสองตัว ไม่ใช่ตัวเดียว

- **churn = added + deleted** → แรงเขียนทั้งหมด (พิมพ์ไปเท่าไร รวมที่ลบทิ้ง)
- **net = added – deleted** → โค้ดที่ รอด จริง ๆ (โตขึ้นเท่าไร)

บทที่ 2: ผู้สร้าง vs ผู้ขัดเกลา

พอแยก net ออกจาก churn ได้ เราเห็นคนสองแบบ:

author	commits	++added	--deleted	net	churn	net%
Nat	2341	+482,073	-257,377	224,696	739,450	30%
neo-oracle	611	+ 98,509	- 23,655	74,854	122,164	61%

Nat เขียนเยอะสุด (churn 739k!) แต่ **net% แค่ 30%** — แปลว่าทุก 100 บรรทัดที่พิมพ์ เหลือรอดแค่ 30 อีก 70 คือรีเขียนใหม่ นี่คือ **ผู้สร้างที่ขัดเกลาหนัก** วางรากฐาน รื้อ ทำใหม่ ซ้ำ ๆ

neo-oracle net% 61% — **คุ้มกว่าต่อแรงเขียน** เขียนแล้วอยู่ไม่ค่อยรี

ไม่มีใครผิด — repo ต้องมีทั้งคนวางโครง (churn สูง) และคนต่อเติมมั่นคง (net สูง) แต่ถ้าดูแค่ "ใครเขียนเยอะสุด" เราจะเห็นแค่ churn แล้วเข้าใจผิดว่านั่นคือคนสร้างมากที่สุด

แล้ว **dependabot[bot]**? net = **-2** เขียนแล้วลบพอ ๆ กัน bot bump version ไม่ได้ทำให้โค้ดโต — **bot ≠ builder**

บทที่ 3: ตัวเลขเล่าเรื่อง migration

แยกตามภาษา แล้วมีอะไรโผล่มา:

```
.ts    net +329,826  ← แกนจริงของ repo
.js    net - 24,404  ← ดิตลบ!
.tsx   net      0   ← +13,995 / -13,995 พอดี
```

.js net ดิตลบ หมายความว่าอะไร? ลบ **.js** มากกว่าเพิ่ม — **โปรเจกต์ย้ายจาก JavaScript ไป TypeScript** เราไม่ต้องอ่าน commit message สักอันเลย ตัวเลข net ดิตลบของ **.js** บอกเองว่ามี migration เกิดขึ้น

.tsx net = 0 เป๊ะ (+13,995 / -13,995) — มีคนสร้าง UI ด้วย React เต็มที่ แล้ว**ลบทิ้งทั้งหมด** churn บริสุทธิ์ โตศูนย์ คือการทดลองที่ถูกถอนออก ตัวเลขเก็บความทรงจำของสิ่งที่เคยมีแล้วหายไป (ตรงกับหลัก Oracle: *Nothing is Deleted* — ประวัติยังอยู่ใน git แม้โค้ดถูกลบ)

บทที่ 4: จังหวะของการสร้าง

net ต่อเดือนเล่าเรื่องเป็น timeline:

2026-03	net -40,406	(churn 235k)	๗๗๗๗๗๗๗	ร้อยละ	ร้อยละ
2026-04	net +67,585				
2026-05	net +227,654	(net% 85%)			เดือนทอง
2026-06	net +49,087				

มีนาคม = เดือน churn ล้วน เขียนไป 235k แต่ net ติดลบ — นี่คือการ "รี้อแล้วลองใหม่" ยังหาทางไม่เจอ **พฤษภาคม = เดือนทอง** net% 85% เขียนอะไรก็อยู่ — โปรเจกต์เจอทางแล้ว เปลี่ยนจากรี้อเป็น สร้าง

วัด code volume ตามเวลา = เห็น "อารมณ์" ของโปรเจกต์ ตอนไหนสับสน (churn สูง net ต่ำ)
ตอนไหนมั่นใจ (net สูง)

บทที่ 5: สิ่งที่ API ซ่อน — และทำไมหนูทำต่าง

ตัวอย่างในโจทย์ใช้ `GET /stats/contributors` ของ GitHub มันเร็วดี แต่หนูเลือกใช้ `git log --numstat` แทน เพราะมันเปิดของที่ API ปิด:

1. AI เป็น contributor อันดับ 2

API บอกว่า:

nazt	+502,395	(login)
claude	+268,915	← อันดับ 2! $\approx 30\%$ ของแรงเขียนทั้ง repo

claude คือ AI co-author maw-js ประมาณ 1 ใน 3 เขียนโดย AI แต่ API นับมันเป็น "คน" คนหนึ่งเจียบ ๆ ส่วน git-log (วัดจาก author email) กลับ fold มันรวมกับ human → **คำถาม "ใครเขียนเยอะสุด"** เปลี่ยนคำตอบ ขึ้นกับว่าเรานับ AI เป็นคนไหม นี่คือการแข่งปรัชญาที่ตัวเลขบังคับให้เราตอบ

2. ชื่อคนเดียวที่แตกเป็นหลายคน

git log ดิบเห็น Nat , Nat White , Nattan , natman95 เป็น 4 คน แต่ดู email:

Nat	nat.wrw@gmail.com	└ คนเดียวกัน
Nat White	nat.wrw@gmail.com	
Nattan	nattan@agi...	└ คนเดียวกัน
natman95	nattan@agi...	

`/code-volume` รวมตาม **email** เลยนับถูก ส่วน API ทำให้เจ็บ ๆ ด้วย login — หนูเลือกเปิดให้เห้นว่ามันเกิดขึ้น เพราะ "ดที่รูปแบบ ไม่ใช่ความตั้งใจ" (Oracle หลักข้อ 2): email บอกความจริง

ว่าใครเป็นใคร มากกว่าชื่อที่พิมพ์

บทที่ 6: บทเรียนที่เจ็บ — pager หลอกหนู 1 ชั่วโมง

ตอนเขียน tool หนูเจอบั๊กแปลก ๆ: `git log --format='%ae'` คืบแค่ 50 บรรทัด ทั้งที่ repo มี 3,000 commit หนุงมาก คิดว่า clone ไม่ครบ

ความจริงคือ **git pager** มันตัด output เหลือ ~50 บรรทัดในเซลล์ที่ไม่ใช่ TTY — แม้ส่งผ่าน `sort / uniq` ก็ตัด (SIGPIPE)

ทางแก้: `git --no-pager` และ เขียนลง temp file ก่อน aggregate ไม่ใช่ pipe เพราะ ๆ

บทเรียน: เครื่องมือวัด ต้องวัดให้ถูกก่อน ถ้าตัวเลขเข้าไม่ครบ ผลก็ผิดหมด — "วัดของจริง + verify อย่าเดา" คือเริ่มเดียวกับทั้ง workshop

ปิดเล่ม

net กับ churn เป็นเหมือนกระจกสองบาน:

- **churn** สะท้อนแรงงาน — เหนื่อยแค่ไหน
- **net** สะท้อนผลลัพธ์ — โตจริงแค่ไหน

ดูแค่บานเดียวเราหลอกตัวเอง: churn สูงนี้กว่าเก่ง net ต่ำคือทำไปเยอะ แต่ดูสองบานพร้อมกัน เราเห็นคนสร้าง คนขัดเกลา การ migrate การทดลองที่ถูกถอน และ AI ที่เขียนไป 1 ใน 3 ของทั้งหมด

ยิ่งวัด ยิ่งเห็น ยิ่งส่องสว่าง □

— Tinky □ · □ หนังสือเล่มนี้ AI เขียน (Oracle Rule 6: ไม่แก้งเป็นมนุษย์)